ZOIKO SHIELD PLATFORM

Backend Microservices Architecture & Pipeline Specification

6 Microservices Topology • Preserved Ingestion Normalization • 8-Stage Event Pipeline • 226 Email Templates • Post-Quantum Cryptography

1. Monorepo Microservices Architecture (6 Applications)

The Zoiko Shield backend is a distributed, high-concurrency microservices architecture built with NestJS, Fastify, TypeScript, Kafka, and PostgreSQL. It enforces clean domain boundaries across 6 dedicated applications, ensuring isolation between ingestion, detection, AI reasoning, response execution, and cryptographic ledger attestation.

Application	Domain Responsibility	Key Codebase Path	Critical Security Invariant
shield-core	Identity, Cedar RBAC/ABAC, Multi-Tenancy, 226 Email Engine, Notifications, Rules.	backend/apps/shield-core	Gate 3 Tenant Isolation; Fail-closed secret scanner.
shield-ingest	Connector fleet, high-throughput sharding, OCSF normalization, DLQ retry.	backend/apps/shield-ingest	apps/shield-ingest/src/normalization/ PRESERVED UNTOUCHED.
shield-ai	Vertex AI SecOps Copilot, Hallucination bounds, §16.1 Decision Envelopes.	backend/apps/shield-ai	Model Armor screening; Mandatory human ratification.
shield-action	SOAR response actuator, Cloud Armor sync, emergency freeze, rollback broker.	backend/apps/shield-action	Guaranteed idempotent compensating rollback.
shield-anchor	Post-quantum Merkle trees (Dilithium3 + Ed25519), Sigstore Rekor witness.	backend/apps/shield-anchor	Domain-separated ZS-MERKLE-V1; Cloud KMS HSM signing.
verifier-cli	Zero-network standalone offline cryptographic evidence verifier (ADR-01).	backend/apps/verifier-cli	Fails-closed (Exit Code 1) on single-bit tampering.

STRICT SYSTEM INVARIANT — Strict Ingestion Normalization Preservation Mandate

CRITICAL ARCHITECTURAL BASELINE: All normalization mapping code located in apps/shield-ingest/src/normalization/ represents audited, standardized OCSF v1.1.0 ground truth. It must NEVER be mutated, refactored, or overridden. Every downstream consumer (rules, AI Copilot, SOAR) relies on this immutable contract.

2. End-to-End Real-Time Event Pipeline (8 Stages)

Every security event ingested into the Zoiko Shield platform follows a deterministic, 8-stage pipeline from raw log to Merkle seal:

Stage 1: Multi-Cloud Telemetry Ingestion (shield-ingest)

Connectors stream events from AWS CloudTrail, CrowdStrike, Okta, and Sentinel into Kafka topic `telemetry.raw`. Idempotency keys prevent duplicate processing. Dead-letter queues (DLQ) capture malformed events with automated retry (OPS-INV-13).

Stage 2: Strict OCSF Normalization (Untouched & Preserved)

Raw logs are transformed into standardized OCSF v1.1.0 schemas (Security Finding, Network Activity, Authentication). Normalization algorithms in apps/shield-ingest/src/normalization/ remain strictly preserved without mutation.

Stage 3: Sigma & YARA-L Stream Detection (shield-core)

High-throughput rule evaluators execute across normalized streams with sub-50ms latency. Correlated findings generate structured incident candidates with MITRE ATT&CK; technique tags.

Stage 4: Vertex AI Copilot Reasoning & Decision Envelope Synthesis (shield-ai)

Vertex AI evaluates incident severity, lateral movement paths, and containment strategies. In strict accordance with Spec §16.1, it synthesizes a 10-field Decision Envelope with calibrated confidence intervals and compensating rollback steps.

Stage 5: Cedar Policy Authorization & Dual-Custody Approval (shield-core)

Cedar policy engines validate that proposed response actions adhere to tenant boundaries and operator privileges. Destructive actions mandate 4-eyes dual-custody approval before execution authorization is granted.

Stage 6: Idempotent SOAR Action Execution & Rollback (shield-action)

Containment actions (e.g. endpoint isolation, token revocation, firewall rules) execute idempotently. Compensating actions are staged simultaneously, guaranteeing immediate rollback if verification checks fail.

Stage 7: Post-Quantum Cryptographic Merkle Anchoring (shield-anchor)

State transitions and decision envelopes are hashed into domain-separated Merkle trees (ZS-MERKLE-V1). Epoch roots are anchored to Sigstore Rekor transparency logs and dual-signed with Cloud KMS / Cloud HSM using Dilithium3 + Ed25519.

Stage 8: Enterprise Transactional Email Dispatch (ZS-EML-TPL-001 v2.0)

Security events trigger contractual transactional emails across 226 templates in 16 domains. Gate 3 enforces tenant isolation; Gate 9 routes through 8 reputation sender pools; Gate 6 scrubs secret tokens; and SHA-256 digests provide permanent render receipts for offline audit attestation.

3. Production Transactional Email Matrix (ZS-EML-TPL-001 v2.0)

Zoiko Shield implements the full 226-template transactional email catalog across 16 domains. Every template adheres to the universal render contract: (1) escapes typed variables, (2) uses allowlisted enums for status labels, (3) renders multipart HTML + plain-text, (4) enforces HTTPS CTAs, and (5) computes deterministic SHA-256 render digests.

Domain Code	Domain Name	Count	Gate & Sender Class	Operational Scope
SEC	Security & Casework	19	P0 • security_sender	Critical alerts, escalations, war rooms, post-incident reviews.
IAM	Identity & Access	23	P0 • account_sender	Passkeys, MFA enrollment, password reset, JIT elevation.
ORG	Organization Onboarding	13	P0 • account_sender	Tenant workspaces, domain verification, residency cell changes.
CONN	Connectors & Ingestion	15	P0/P1 • security_sender	Connector drift, DLQ surge, ingestion delay, rate limiting.
POL	Policy Lifecycle	12	P0 • security_sender	Cedar policy approval, dual-custody review, canary rollout.
ACT	Response Actions	13	P0/C • security_sender	SOAR action approvals, emergency freeze, compensating rollback.
INC	P0 Incident Command	16	P0 • security_sender	Formally declared incidents, stakeholder updates, resolution.
REP	Threat Reports & SLA	12	P1 • security_sender	Executive risk digests, scheduled SLA reports, MITRE coverage.
BIL	Commercial & Billing	18	P0 • billing_sender	Invoices, payment receipts, dunning notices, quota limits.
SUB	Subscription Lifecycle	10	P0 • billing_sender	Tier transitions, renewals, seat allocations, cancellations.
AUD	Audit Ledger & Evidence	14	P0/P1 • assurance_sender	Merkle epoch seals, witness receipts, auditor dossier exports.
DIS	Regulatory Disclosures	12	P0 • privacy_sender	DORA reporting, GDPR data subject rights, legal hold notices.
DRI	DR & Failover Drills	12	P1 • internal_ops_sender	Regional cell failovers, backup verification, RTO/RPO metrics.
OFF	Tenant Offboarding	8	P0 • account_sender	Data destruction milestones, cryptographic key purging.
OPS	Internal SRE Operations	20	P0 • internal_ops_sender	Platform incidents, KMS signing failures, write durability.
GOV	Governance & Terms	8	P1 • account_sender	Terms updates, subprocessor notices, deprecation deadlines.

4. Offline ADR-01 Cryptographic Verifier (verifier-cli)

ZERO-NETWORK AUDITABILITY — Why We Have Used Offline Cryptographic Verification

Regulatory compliance frameworks (SOC 2, ISO 27001, DORA) require that audit logs cannot be modified even by cloud administrators. If verification requires querying the cloud provider, a compromised cloud environment could forge the verification responses. Zoiko Shield solves this through the standalone verifier-cli (ADR-01), which reconstructs the Merkle root independently using zero network calls. In automated tamper injection tests, altering a single byte in an audit event causes the verifier to fail-closed with Exit Code 1, proving 100% cryptographic integrity.

A. Mathematical Verification Algorithm

1. **Package Loading:** The verifier loads manifest.json and local event receipt array.
2. **Canonical Serialization:** Each event is canonically sorted and stripped of mutable transport metadata.
3. **Leaf Hashing:** H_leaf = SHA-256('ZS-LEAF-V1' || Canonical_JSON).
4. **Merkle Tree Folding:** Leaves are paired and folded into the intermediate tree nodes: H_node = SHA-256('ZS-NODE-V1' || H_left || H_right).
5. **Root Attestation:** The computed root is compared against the Sigstore Rekor witness hash.
6. **Tamper Detection Rejection:** If any byte mismatches, the verifier prints a tamper warning and exits with code 1.